

RF3 / DEVELOPMENT GUIDE

Firmware roadmap: each phase explained

September 7, 2026 | Scope, implementation sequence, and acceptance gates

The intended product lets a user import an arbitrary binary file through an offline web page, store it on an ESP32, transfer it over RF to another board, and export the received file through a browser. Both boards should support sending and receiving. Files must survive that round trip unchanged, and progress must distinguish bytes sent from files actually verified and published.

Current position

Phase 1 is implemented locally on **codex/phase-one-reliability**, commit **6ba0315**, starting from **aa911c3**. It has not been merged into the active checkout. The current code pinout is authoritative and was preserved. Phases 2 through 7 below are proposed work, not completed features or measured hardware results. Creating this guide makes no firmware changes.

Phase	Primary outcome	Page
1. Reliability fixes	Accurate faults, safe replay handling, responsive status.	2
2. Hardware baseline	Evidence from real boards and repeatable RF measurements.	3
3. Shared filesystem service	Consistent, recoverable file operations and ownership.	4
4. Browser import/export	Offline file management with bounded streaming.	5
5. End-to-end RF release	Complete browser-to-browser file delivery through RF.	6
6. Measured optimization	Faster operation with demonstrated reliability retained.	7
7. Advanced capabilities	Versioned metadata, resume, and justified expansion.	8

How the phases depend on each other

The main sequence is reliability, hardware evidence, storage, browser access, end-to-end qualification, optimization, then extensions. Host-side storage work can proceed while a hardware problem is investigated. Browser prototypes can use agreed storage contracts, but Phase 5 release acceptance requires real-board results. A blocked board test is not evidence that the corresponding gate has passed.

Common delivery rule

Each phase should produce a reviewable commit, focused regression coverage, reproducible evidence, and a short handoff describing remaining limits. Agree on test conditions before measuring performance. Continue using isolated branches/worktrees until changes are deliberately integrated. This guide does not authorize merging or flashing.

Basis: the project audit, the agreed seven-phase plan, and the committed Phase 1 handoff at docs/phase_one_reliability.md on the isolated branch identified on page 2.

PHASE 1 / IMPLEMENTED LOCALLY

Fix the reliability baseline

This phase removes misleading outcomes and recovery defects before adding more ways to move files. A successful API response must correspond to the operation that actually happened. It also establishes status reporting that remains useful while the radio is occupied.

1. Detect communication faults accurately

Invalid SPI STATUS/FIFO values previously could be interpreted as TX success. The driver now rejects impossible snapshots, lowers CE, suppresses fallback transmission on a communication fault, and exposes radio fault 10. Valid zero STATUS is preserved because zero alone is not sufficient proof of disconnection. Local radio success still does not mean that the receiving board has published a complete file.

2. Preserve cleanup outcomes

Cancellation, timeout, and other receiver failures now reply with the actual post-cleanup error. A failed partial-file removal is visible to the peer. Repeated CANCEL retries failed cleanup; successful cancellation remains idempotent, meaning repeating it does not create an additional operation or contradictory result.

3. Handle delayed completed-session traffic

A fixed-memory cache remembers four successful completions. Matching delayed START packets receive COMPLETE without reopening a file, replacing a newer session, or emitting another publication event. Reusing a remembered ID with different metadata is rejected. Explicit successful reset, reboot, and cache eviction bound this protection; it is not persistent replay history.

4. Make status safe and responsive

Serial and HTTP status now copy owned snapshots under a separate mutex. They do not wait for an entire RF transfer or borrow a filename that another command can invalidate. The sender publishes progress during transfer; the worker refreshes idle diagnostics every 250 ms when it can acquire the radio lock. Text is JSON-escaped, and progress identifies preparation, transfer, errors, and bytes per second.

Deliverables and completed validation

The branch also removes the machine-specific documentation path that failed repository hygiene. Validation passed 211 native tests, 16 Python tests, and 264 qualification cases, plus both canonical firmware and filesystem builds, a Wi-Fi-enabled HTTP compile/link check, and offline handoff checks. Six protocol regressions failed against the original header before passing after the fix. Host checks establish software behavior, not physical RF reliability.

Gate and next dependency

The implementation and software gate are complete on the isolated branch. Review and integration are still separate decisions. Phase 2 must test live task scheduling, status responsiveness, SPI failures, and RF behavior. Keep the existing pinout and wire format while establishing that baseline.

Commit: 6ba0315276a46c27e15a4b7cde044c1c8764d14c

Branch: codex/phase-one-reliability

Start here on another computer: docs/project_handoff.md

Implementation details and reproducible commands: docs/phase_one_reliability.md.

PHASE 2 / PLANNED

Establish the hardware baseline

The purpose is to replace assumptions about the board, radio, and timing with repeatable observations. Simulation can verify protocol logic, but cannot establish power stability, radio range, actual goodput, flash latency, or how the ESP32 tasks behave together.

Entry conditions and bench record

Use a reviewed Phase 1 build and the correct existing pin profiles. Positively identify each board and its flash layout before any separately approved flashing. Record firmware revision, board identity, power source, channel, power setting, data rate, distance, antenna arrangement, file size, and test direction. These conditions must travel with every performance result.

Work package A: basic board and radio behavior

Check power stability, reset causes, boot behavior, SPI register access, and expected radio state transitions. Investigate the previously reported power instability with actual measurements rather than compensating with longer software delays. Confirm that an absent or disconnected radio produces useful diagnostics and does not become a false successful transfer. Record whether a fault requires an explicit retry, reinitialization, or board reset.

Work package B: file integrity on the air

Start with an empty file and a one-byte file, then representative small and large binary files in both directions. Test the protocol maximum of 1,310,720 bytes only where storage admission permits it. Compare the original and exported files independently, not just the firmware success flag. Repeat transfers without rebooting to expose stale state and resource leaks.

Work package C: controlled failure and responsiveness

Interrupt the peer, cancel a transfer, exercise poor-link conditions, and test recovery after a failed attempt. During long transfers, request serial/HTTP status and record response latency. Check watchdog events, free heap, stack headroom, and whether the next transfer can begin cleanly. Treat a clean, accurately reported failure differently from data corruption or a false completion.

Measurements and deliverables

Separate source preparation, on-air transfer, receiver verification/publication, and total operation time. Record unique payload bytes per second, retries, completion outcome, and observed status latency. Deliver a bench procedure, results dataset, byte-comparison evidence, logs, and a list of reproducible hardware problems. Do not report the host qualification runtime as RF throughput.

Acceptance gate and parallel work

The planned cases must preserve bytes, avoid false completion, and recover as specified. Repeatability must be demonstrated with stated sample sizes and conditions; a single success is insufficient evidence of reliability. Choose numerical performance targets from this baseline. Storage development can continue using host tests during a hardware investigation, but the first end-to-end release remains gated on board evidence.

PHASE 3 / PLANNED

Build the shared filesystem service

Serial commands, browser operations, and RF reception should use one set of storage rules. The service owns file identity, visibility, capacity, access, and cleanup so that each entry point does not invent a different interpretation of a complete file.

Storage contracts and file identity

Define operations for list, metadata, begin import, write, verify, publish, open for export/transmit, cancel, and delete. Use an opaque file ID independently of the display filename and RF transfer ID. Keep physical names within the selected filesystem limits and prevent names from selecting arbitrary paths. Preserve original display names in local metadata when possible; carrying them across RF remains a versioned extension.

Transaction lifecycle

Use **admit - stage - write - close - verify - publish**. Admission checks the declared length, protocol/profile limits, reserved space, and other active operations. Stream through bounded buffers into an unpublished staging file. Verify stored length and integrity before making it visible as a completed file. A completed write call or a file appearing on disk must not by itself mean publication succeeded.

Ownership, scheduling, and capacity

Give open exports and RF sources read leases so a concurrent delete or replacement cannot invalidate them. Start with one active writer and a bounded number of readers; define queue or busy behavior explicitly. If a bulk read jeopardizes RF deadlines, schedule it accordingly while leaving status responsive. Determine the space margin from filesystem behavior and measurements, rather than treating unused partition bytes as guaranteed payload capacity.

Recovery and publication design

Validate the actual filesystem close, rename, and power-interruption behavior before choosing a publication mechanism. If necessary, use a recoverable metadata/commit record so data and catalog state can be reconciled after reboot. Before publication, failures leave the file hidden. After publication, repeated requests resolve to the same file ID. First-release partial uploads may restart after reboot; they must never silently appear complete.

Failure tests and deliverables

Exercise short writes, exhausted space, close failures, cleanup failures, cancellation, duplicate requests, active-file deletion, and reboot at every transaction boundary. After restart, reconcile unfinished staging data and catalog entries without deleting unrelated valid files. Deliver the shared API, state model, locking/lease rules, capacity policy, recovery procedure, and fault-injected tests. Keep a filesystem migration out of this phase unless measurements establish it as necessary.

Acceptance gate and boundary

A failed or cancelled import is never advertised as complete; an existing valid file survives unrelated failures; successful publication is independently verifiable; repeated terminal actions are safe; and memory use stays bounded as files grow. Phase 4 depends on these contracts. Browser controls should consume this service rather than directly constructing filesystem paths or bypassing ownership checks.

PHASE 4 / PLANNED

Deliver browser import and export

The first web release provides offline file management through an ESP32 SoftAP: list, details, upload, download, delete, and status. Package the page and its required assets locally so normal operation needs no router, cloud service, or external asset download. SoftAP file management is future work; the existing optional HTTP control interface is only a starting point.

Import: separate transport progress from completion

Preflight a file name, declared size, and agreed integrity value to obtain an upload/job ID. Send binary content through bounded device buffers, honoring storage backpressure. After the body arrives, the storage service closes, verifies, and publishes the file. The page must distinguish browser bytes sent, board bytes stored, verification, and final completion. Upload progress reaching 100 percent is not permission to announce a verified file.

A concrete API shape to agree before coding

One proposed sequence is POST /api/uploads for admission, PUT /api/uploads/{id}/content for a known-length binary body, and POST /api/uploads/{id}/commit for final verification/publication. GET /api/jobs/{id} reports progress; DELETE /api/uploads/{id} cancels. These are proposed contracts, not existing endpoints. Make repeated commit/cancel requests idempotent and define whether each result is immediate or accepted for asynchronous work.

Export and deletion

GET /api/files/{id}/content should stream a stable leased file with its length and safe download name. Prefer browser-managed downloads over assembling the entire file in page memory. Release the lease after completion or disconnect. A delete request must honor active readers/writers and report a clear result. The device can report stream progress; it cannot assume the browser has saved and independently verified a local disk file.

Keep the page responsive

Long uploads/downloads must not monopolize the HTTP request-processing task. Select a supported asynchronous handoff or worker arrangement and make request lifetime/response ownership explicit. Status reads use cached owned data; handlers do not hold the radio lock while waiting for browser traffic. Define reconnect behavior, job ownership, queue limits, and allowed clients. Keep UI assets outside user-deletable file entries.

Tests, deliverables, and acceptance

Deliver the offline page, documented API, streaming handlers, job states, validation rules, and user-visible recovery messages. Test disconnects mid-body, wrong lengths/checksums, low space, duplicate names, unusual characters, repeated actions, and simultaneous status requests. Reboot recovery follows Phase 3; resumable upload is deferred. Accept this phase when uploads and exports preserve bytes, memory remains bounded, partial files stay hidden, and control requests remain responsive during bulk operations.

Dependencies: Phase 3 storage contracts and Phase 1 status snapshots. RF transmit controls and complete two-board workflow qualification belong to Phase 5.

PHASE 5 / PLANNED

Complete the end-to-end RF release

This phase joins the local browser file manager to the proven RF subsystem. The release workflow is browser A - board A storage - RF - board B storage - browser B. A local upload and a successful RF packet send are intermediate states; the receiving board must verify and publish the complete file before the transfer is reported complete.

Integrate commands through one operation model

Add select/transmit actions to the page and use stable source file IDs. Create a transfer job that binds the source lease, RF transfer ID, total length, integrity value, and progress state. Keep upload, RF transfer, and download job identities distinct so the page cannot display progress from a previous operation. Either board should be usable as sender or receiver.

Expose the complete lifecycle

Show preparation, peer handshake, data transfer, remote verification/publication, completion, cancellation, and failure. Include useful progress when retries occur and distinguish temporary waiting from a terminal error. Cancellation must report the cleanup outcome and release storage ownership. A lost browser connection should not accidentally start a second RF session when the user reconnects.

Respect existing wire compatibility

Use the current protocol until an explicit versioned extension is introduced. Current RF metadata does not carry the original filename, so the receiver-assigned name must be visible and unambiguous. Do not pretend the original filename survived the link. Expose completed received files through the same file list and export path as local imports; do not invent a second publication rule for RF files.

Define coexistence and recovery

Agree how new uploads, exports, deletes, and RF jobs compete for storage and radio access. Reject or queue conflicting operations with a useful explanation while keeping status available. Test retrying after peer restart, user cancellation, cleanup failure, stale control packets, and a lost terminal response. Reconnecting clients should query the existing job/file state instead of guessing completion from a closed connection.

Release qualification

Run complete round trips with empty, one-byte, representative, and maximum-admitted binary files in both directions. Independently compare original and final browser exports. Repeat back-to-back operations, poor-link transfers, cancellation at each stage, storage exhaustion, and concurrent status activity. Measure total user-observed time as well as radio time, and retain evidence for every failed case.

Deliverables and acceptance gate

Deliver the integrated browser controls, job mapping, two-board operating procedure, release candidate, and reproducible qualification record. Acceptance requires verified bytes, no false completion, recoverable failures, responsive status, and completed Phase 2 hardware gates. This is the first complete product release. Later optimization should use this release as its comparison baseline rather than changing several subsystems at once.

PHASE 6 / PLANNED

Measure and optimize performance

Improve the slowest measured parts of the complete workflow while retaining the established integrity and recovery guarantees. A faster synthetic loop is useful only if it improves the real user operation without creating new failure modes.

Metric	Meaning
Payload goodput	Unique accepted/acknowledged payload bytes divided by a named interval; label units as bytes per second.
First-attempt exchange success	Logical exchanges completed without retry divided by attempted logical exchanges. This is not direct physical packet loss.
Retry distribution	Retries per exchange/transfer, plus exhaustion counts and failure reasons.
Latency and stage timing	Preparation, handshake, data, verification/publication, status response, and total completion time.
Resource behavior	Heap, task stack headroom, queue occupancy, storage stalls, and reset/watchdog events.

Instrumentation first

Keep counters and bounded timing summaries close to the operations they describe. Publish snapshots for the UI and export structured results with firmware revision and bench conditions. Avoid logging every packet in the timing-sensitive path. Explain whether an interval includes preparation and retries; otherwise two throughput figures may measure different things.

Optimization sequence

First make source preparation/CRC scanning cancellable and measure its cost. Then reduce repetitive diagnostic SPI polling where safe, tune buffer sizes and storage scheduling, and measure the cost of status publication. Preserve authoritative FIFO checks and radio serialization. Consider interrupt-driven servicing with a fallback only after the current behavior has a reproducible baseline.

How to judge a change

Change one major variable at a time and compare the same files, board pair, power setup, and link conditions. Include weak-link and failure cases, not only the fastest healthy run. Inspect latency tails and cancellation responsiveness alongside averages. Keep or revert a change based on documented improvements and unchanged integrity/recovery outcomes.

Deliverables and gate

Deliver a before/after dataset, exported metrics, benchmark procedure, resource budgets, and a separate reviewable commit for each substantial optimization. Rerun the relevant regression and end-to-end cases after changes. Do not loosen retry, size, or validation rules simply to make a benchmark look faster. Protocol windowing, persistent resume, and storage-format migration belong to Phase 7 when the evidence justifies their cost.

PHASE 7 / PLANNED / REQUIREMENT-DRIVEN

Add advanced capabilities deliberately

This phase is a set of optional extensions, not a single mandatory rewrite. Prioritize the capability that solves an observed limitation of the Phase 5 release. Each extension needs its own compatibility rules, resource budget, failure behavior, tests, and rollback/integration plan.

7A. Preserve filenames and richer metadata

Add an explicitly versioned RF metadata exchange for logical filenames and any needed content information. Specify encoding, length bounds, sanitization, collision handling, and fallback behavior for older peers. Keep file identity separate from display names. Test mixed firmware versions, malformed metadata, maximum names, and an interrupted metadata exchange before enabling this as the default.

7B. Resume browser transfers

Add upload/download offsets only after identifying the same immutable file or upload session reliably. Define which acknowledged offsets are durable, how partial content is verified, what expires, and what survives reboot. Replayed chunks must not append duplicate bytes or overwrite unrelated files. Test disconnects and resets around every acknowledged boundary. Restarting from zero remains the supported fallback.

7C. Resume or pipeline RF transfers

RF resume requires durable receiver progress and agreement on source identity. Windowed transmission additionally requires bounded buffering, acknowledgment rules, duplicate handling, out-of-order policy, backpressure, and sequence-wrap rules. Model loss/reordering before board testing. Extend the existing fault campaigns and compare memory, goodput, and recovery against stop-and-wait behavior under both strong and weak links.

7D. Larger files or another filesystem

Larger files affect packet counts, sequence widths, integer bounds, free-space admission, verification time, and recovery metadata together. Change those limits coherently through a compatible protocol version. Evaluate a filesystem migration with measured need, representative workloads, and power-interruption tests. Plan how existing user files and rollback are handled before changing the on-flash format.

7E. Bluetooth and other interfaces

Add another interface only for a defined user need after browser/RF behavior is stable. Reuse the same storage and job services rather than duplicating transfer logic. Define command ownership and test coexistence, scheduling, memory use, and cancellation across interfaces. Introducing another control path must not bypass file leases or verification/publication rules.

Completion criteria and order

A practical order is filename metadata, browser resume, then whichever RF/storage limitation measurements identify. Bluetooth remains optional. Accept each extension only with bounded resources, verified bytes, predictable recovery, and demonstrated behavior with supported older peers. Update the operating procedure and evidence record for every release; do not label the whole firmware mature because one advanced feature works.

The near-term priority remains the real-board baseline and first reliable browser-to-browser release. Phase 7 choices should follow observed requirements, not delay that release.